

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Лабораторная работа №4 «Развертывание приложений на
удаленном сервере»**

Выполнила:

Клапнева Диана

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- провести предварительную настройку удалённого сервера (установка необходимых зависимостей);
- провести развёртывание приложения на удалённом сервере;
- настроить nginx в режиме обратного прокси для обеспечения доступа к вашему приложению извне.

Ход работы

Сначала пересоберем наши сервисы. Type script compiler переводит код на javascript тк node.js его не понимает.

```
C:\Users\local\Documents\GitHub>cd user-service && npm run build

C:\Users\local\Documents\GitHub\user-service>cd ../estate-service && npm run build
> tsc
> estate-service@1.0.0 build
> tsc

C:\Users\local\Documents\GitHub\estate-service>cd ../deal-service && npm run build
> deal-service@1.0.0 build
> tsc

C:\Users\local\Documents\GitHub\deal-service>cd ../messaging-service && npm run build
> messaging-service@1.0.0 build
> tsc

C:\Users\local\Documents\GitHub\messaging-service>cd ../api-gateway && npm run build
> api-gateway@1.0.0 build
> tsc

C:\Users\local\Documents\GitHub\api-gateway>
```

Создадим виртуальную машину на базе Yandex Cloud. Здесь также важно настроить правила трафика для безопасности – определяем, откуда можно подключаться к серверу.

Добавление правила

✕

Направление трафика

↘ Входящий

↗ Исходящий

Диапазон портов

80

Выбрать все

Протокол

Любой

▼

Источник

Диапазон адресов

▼

IPv4 CIDR * ?

0.0.0.0/0

🗑

Добавить CIDR

Описание

HTTP

Отменить

Сохранить

Правила для входящего трафика 9 + Добавить правило

Источник ↑↓	Протокол ↑↓	Порты ↑↓	Описание ↑↓	⚙
Префикс 0.0.0.0/0	Any	—	—	...
Префикс 0.0.0.0/0	Any	22	SSH	...
Префикс 0.0.0.0/0	TCP	80	HTTP	...
Префикс 0.0.0.0/0	TCP	443	HTTPS	...
Префикс 0.0.0.0/0	TCP	3001	User Service	...
Префикс 0.0.0.0/0	TCP	3000	API Gateway	...
Префикс 0.0.0.0/0	TCP	3002	Estate Service	...
Префикс 0.0.0.0/0	TCP	3003	Deal Service	...
Префикс 0.0.0.0/0	TCP	3004	Messaging Service	...

✓ Правило без проблем

Оценить

Подключаемся к виртуальной машине с помощью SSH-ключа.

У меня на пк с Ubuntu пока что нет node.js и postgres, поэтому их мы также установим с остальными зависимостями.

```
sudo apt update && sudo apt upgrade -y
```

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

```
sudo apt-get install -y postgresql postgresql-contrib  
sudo systemctl start postgresql  
sudo systemctl enable postgresql
```

```
sudo apt-get install -y nginx
```

```
sudo npm install -g pm2
```

Создадим базы данных users_db, estates_db, deals_db, messages_db.

Далее копируем файлы проекта на сервер.

```
scp -r user-service student@158.160.224.131:~/apps/  
scp -r estate-service student@158.160.224.131:~/apps/  
scp -r deal-service student@158.160.224.131:~/apps/  
scp -r messaging-service student@158.160.224.131:~/apps/  
scp -r api-gateway student@158.160.224.131:~/apps/
```

Для каждого сервиса устанавливаем зависимости через npm install -> npm run build.

Запустим каждый сервис через PM2 – Process Manager 2 для управления проектами node.js

```
pm2 start npm --name "user-service" -- run start -- --port 3001  
pm2 start npm --name "estate-service" -- run start -- --port 3002  
pm2 start npm --name "deal-service" -- run start -- --port 3003  
pm2 start npm --name "messaging-service" -- run start -- --port 3004  
pm2 start npm --name "api-gateway" -- run start -- --port 3000  
  
pm2 save  
pm2 startup|
```

Создадим файл по пути /etc/nginx/sites-available/api-gateway, в нем будет моя конфигурация (тут все возможные конфигурации, а активные будут отображены в sites-enabled)

```

server {
    listen 80;
    server_name 158.160.224.131;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api-docs/users {
        proxy_pass http://localhost:3001/api-docs;
        proxy_set_header Host $host;
    }

    location /api-docs/estates {
        proxy_pass http://localhost:3002/api-docs;
        proxy_set_header Host $host;
    }

    location /api-docs/deals {
        proxy_pass http://localhost:3003/api-docs;
        proxy_set_header Host $host;
    }

    location /api-docs/messaging {
        proxy_pass http://localhost:3004/api-docs;
        proxy_set_header Host $host;
    }

    location /health {
        proxy_pass http://localhost:3000/health;
        proxy_set_header Host $host;
    }
}

```

Через curl `http://localhost:3000/health` проверяем статус удаленного сервера.

```

{"service":"api-gateway","status":"OK","timestamp":"2026-08-13T14:30:45.123Z",
"port":3000}

```

Вывод

Данная лабораторная работа позволяет познакомиться с принципом реализации развертывания приложения на удаленном сервисе, что приближено к реальным проектам.